

Why Documentation Can Be Correct and Still Fail Developers

Documentation Drift, Developer Experience, and Semantic Fidelity

Reality Drift Framework — Short Concept Note #4

A. Jacobs

The Basic Pattern

Documentation is created to transfer understanding.

An API reference explains system behavior.

A tutorial explains a workflow.

A runbook explains how to respond to failure.

An architecture diagram explains how components relate.

At first, these representations remain close to the system developers actually encounter.

The examples work.

The concepts match practice.

The documented path is the real path.

Over time, the documentation can remain technically correct while becoming less capable of helping someone understand or use the system.

The core sequence is:

working system → documented representation → system evolution → context loss →
documentation drift

Documentation Does Not Store Understanding

Documentation stores representations of understanding.

A reference page represents an interface.

A code sample represents an implementation path.

A diagram represents an architecture.

A troubleshooting guide represents operational experience.

These representations make technical knowledge transferable across people, teams, and time.

But they do not preserve the full understanding that produced them.

They rarely contain every assumption.

Every exception.

Every dependency.

Every historical reason.

Every practical judgment an experienced developer uses without noticing.

The document remains.

Part of the understanding disappears.

The Difference Between Accuracy and Usefulness

Documentation can be accurate at the level of individual statements and still fail as a guide to the system.

A parameter description may be correct.

The endpoint may behave exactly as documented.

The sample code may compile.

The installation instructions may succeed.

Yet a developer may still be unable to determine:

Which path should be used in a real project?

Which features are stable?

Which combinations create problems?

Which defaults are safe?

Which parts of the system are legacy?

What experienced users avoid?

The documentation contains facts.

It does not necessarily preserve orientation.

Documentation Drift

Documentation Drift occurs when documentation remains available and technically valid while its relationship to the lived reality of using the system gradually weakens.

The pages remain published.

The examples still run.

The references remain current.

The knowledge base continues growing.

But developers increasingly depend on issue threads, private messages, source-code inspection, or experienced colleagues to understand what actually works.

The representation survives.

The operating reality moves elsewhere.

Why Documentation Becomes Less Useful Without Becoming False

Systems rarely change all at once.

They accumulate layers.

New features are added.

Older workflows remain supported.

Temporary workarounds become permanent.

Teams develop local conventions.

Dependencies change.

The ecosystem evolves around the official interface.

Documentation updates usually track visible changes.

They are less capable of tracking the growing structure of practical context around those changes.

The documented facts remain true.

The unstated conditions become more important.

The Happy-Path Problem

Most documentation is organized around intended use.

Install the library.

Authenticate.

Call the endpoint.

Handle the response.

Deploy the application.

This creates a clean path through the system.

Real development is rarely that clean.

Users arrive with existing architectures.

Conflicting dependencies.

Partial permissions.

Legacy data.

Unusual performance requirements.

Organizational constraints.

The happy path may be correct while representing only a shrinking portion of real use.

As the system matures, edge cases stop being peripheral.

They become the environment.

When Examples Preserve an Earlier Reality

Examples are powerful because they compress explanation into action.

A developer can copy the pattern and see the system work.

But examples often preserve the assumptions of the moment in which they were created.

The recommended library changes.

The surrounding framework evolves.

Deployment practices shift.

Security expectations increase.

The code still works.

It no longer represents how competent practitioners would build the same thing today.

The example remains executable.

Its practical meaning has drifted.

Semantic Fidelity

Semantic Fidelity describes how well a representation preserves the meaning, structure, priorities, and constraints of what it represents.

For documentation, semantic fidelity is not limited to factual correctness.

High-fidelity documentation helps the reader build an accurate mental model of the system.

It explains not only what a feature does, but when it matters.

How it interacts with other parts.

Which assumptions constrain it.

Where the abstraction breaks.

A document can have high technical accuracy and low semantic fidelity.

Every sentence may be true while the reader forms the wrong model.

Reference Documentation and Mental Models

Reference documentation answers local questions.

What arguments does this function accept?

What does this field contain?

Which status codes can appear?

Developers also need a global model.

How does the system fit together?

Where does state live?

Which operations are reversible?

Which components are authoritative?

Where do failures propagate?

A system can have excellent reference coverage and weak conceptual documentation.

The reader can find every component without understanding the whole.

Completeness at the page level can coexist with confusion at the system level.

Documentation as Interface

Documentation is often treated as supporting material around the product.

For developer tools, documentation is part of the product interface.

A developer cannot directly inspect the full system.

They encounter it through names, examples, guides, schemas, errors, and explanations.

Documentation mediates the relationship between the developer and the underlying technology.

When that mediation weakens, the system can remain functional while becoming progressively harder to use correctly.

The software works.

The path to understanding it does not.

The Informal Knowledge Layer

When official documentation loses contact with practice, an informal knowledge layer emerges.

Developers search issue trackers.

Read old forum answers.

Inspect tests.

Follow unofficial tutorials.

Ask maintainers directly.

Copy patterns from working repositories.

This layer often contains the context missing from official documentation.

It also fragments understanding across unstable sources.

The system may have extensive documentation while its most valuable knowledge exists outside the documentation system.

The official representation remains visible.

The practical representation becomes distributed and difficult to govern.

Why More Documentation Can Make the Problem Worse

A common response to documentation failure is to produce more documentation.

More guides.

More examples.

More pages.

More generated reference material.

This can improve coverage.

It can also increase navigational burden.

Developers must determine which page is current.

Which example applies.

Which guide reflects the recommended path.

Which explanation is authoritative.

The documentation system becomes larger than any reader can comprehend.

The information exists.

The cost of locating and interpreting the right representation increases.

Documentation accumulation can begin replacing documentation clarity.

AI-Generated Documentation

AI makes documentation easier to produce.

It can generate reference descriptions.

Explain code.

Create examples.

Summarize changes.

Convert source material into tutorials.

This increases volume and speed.

It also creates a new risk.

Generated documentation can be locally plausible while lacking the practical judgment that distinguishes an accurate explanation from a useful one.

The system may generate more pages than maintainers can validate against real developer experience.

Documentation becomes easier to expand and harder to keep grounded.

Recursive Documentation

Documentation increasingly becomes input for more documentation.

Source code generates references.

References generate summaries.

Summaries generate tutorials.

Tutorials are ingested by AI assistants.

AI responses become new internal guides.

Each transformation compresses and reorganizes the earlier representation.

The original system becomes increasingly distant from the explanation encountered by the developer.

This is a form of **Recursive Compression**.

The developer may be reading a representation generated from a summary of a representation of the system.

Each layer can remain coherent while semantic fidelity gradually declines.

Developer Experience Drift

Developer Experience Drift occurs when the measurable surfaces of developer support remain healthy while practical comprehension deteriorates.

Documentation coverage increases.

Search traffic grows.

Onboarding completion improves.

Support deflection rises.

Time spent on reference pages declines.

These indicators can suggest success.

Developers may still feel that the system is difficult to reason about, unpredictable in practice, or dependent on hidden knowledge.

The experience drifts outside the measurements used to evaluate it.

Why the System Still Looks Healthy

Documentation Drift is difficult to recognize because the visible infrastructure remains operational.

The site is online.

The search function works.

The API reference is generated automatically.

Every public method has a description.

Examples pass automated tests.

Documentation releases accompany product releases.

These indicators show that documentation exists and remains maintained.

They do not show whether it transfers usable understanding.

The system measures publication.

The developer experiences interpretation.

A Recognition Test

Ask:

Can a competent developer choose the correct implementation path without informal guidance?

Do examples represent current best practice or merely code that still works?

Does the documentation explain system behavior, or only describe individual interfaces?

Can developers identify which source is authoritative when pages conflict?

Are recurring support questions treated as user confusion or as evidence that the representation is failing?

Would a new developer form the same mental model as an experienced maintainer after reading the documentation?

These questions test whether documentation remains connected to practical understanding.

Documentation and Reality Drift

Within the **Reality Drift framework**, documentation is a representational layer between a technical system and the people trying to understand it.

The representation allows knowledge to scale beyond direct apprenticeship and individual expertise.

As the system grows, documentation becomes more necessary.

It can also become less capable of preserving the context required for correct interpretation.

Reality Drift begins when the documentation remains coherent, accurate, and operational while becoming less answerable to the realities developers encounter.

The reference remains correct.

The developer still cannot find the truth of the system inside it.

The Core Principle

Documentation succeeds when it transfers a usable model of the system, not merely a collection of correct statements.

Documentation can be correct and still fail developers when factual accuracy survives but practical context disappears.

Documentation Drift begins when the official representation remains stable while the knowledge required to use the system moves into informal practice.

Semantic Fidelity depends on whether documentation preserves the distinctions and constraints developers need to act, not simply whether every sentence is technically true.

The goal is not to document more of the system. It is to keep the documented system answerable to the system developers actually encounter.

Keywords: why documentation fails developers, documentation drift, developer experience problems, API documentation problems, technically correct documentation, documentation accuracy versus usefulness, developer onboarding, technical writing, DevRel, developer education, internal documentation, knowledge bases, runbooks, architecture documentation, code examples, API reference, documentation maintenance, AI-generated documentation, semantic fidelity, recursive compression, developer experience drift, knowledge transfer, Reality Drift, A. Jacobs